

# 软件安全 实验报告

专业名称：软件工程

课程名称：软件安全

任课教师：陈金富

学生学号：2023302111285

学生姓名：李亦凡

**2026 年 4 月**

# 目 录

|                                         |    |
|-----------------------------------------|----|
| 实验一 .....                               | 3  |
| 一、实验目的 .....                            | 3  |
| 二、实验环境与工具 .....                         | 3  |
| 三、PE 文件代码注入 (winmine_patched.exe) ..... | 5  |
| 3.1 样本准备 .....                          | 5  |
| 3.2 入口点 (OEP) 定位与验证 .....               | 6  |
| 3.3 调用链与参数含义 .....                      | 7  |
| 3.4 注入目标与可验证性设计 .....                   | 8  |
| 3.5 注入代码片段 .....                        | 8  |
| 3.6 修改入口点与保存 .....                      | 9  |
| 3.7 运行验证 .....                          | 9  |
| 3.8 现象与原因分析 .....                       | 9  |
| 实验二 .....                               | 10 |
| 四、OllyDbg 调试分析 (CrackLab.exe) .....     | 10 |
| 4.1 生成可执行文件 (VC6 Debug) .....           | 10 |
| 4.2 运行现象验证 .....                        | 10 |
| 4.3 从字符串定位到关键逻辑 .....                   | 11 |
| 4.4 返回值 (EAX) 与条件跳转导致的分支差异 .....        | 11 |
| 4.5 汇编修改前后对比与结果验证 .....                 | 12 |
| 五、遇到的问题与解决方法 .....                      | 12 |
| 六、实验总结 .....                            | 12 |

# 实验一

## 一、实验目的

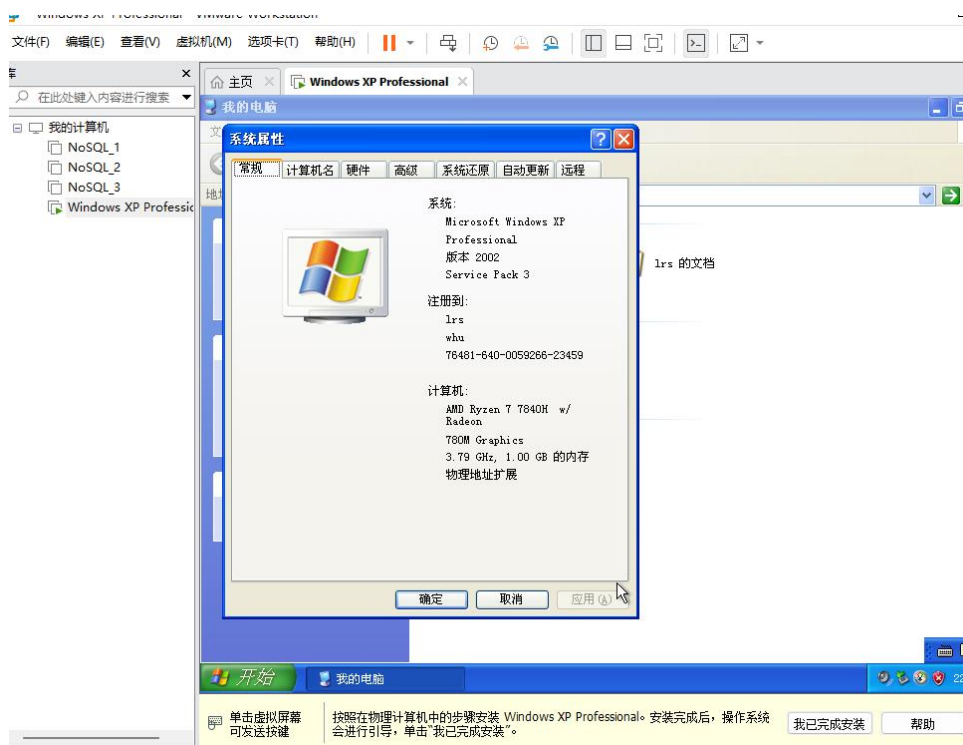
(1) 掌握 PE 文件入口点（OEP）与导入表（IAT）的基本概念，理解程序启动与跨模块 API 调用机制。

(2) 完成一次可验证的 PE 文件代码注入实验，并通过运行现象证明注入成功。

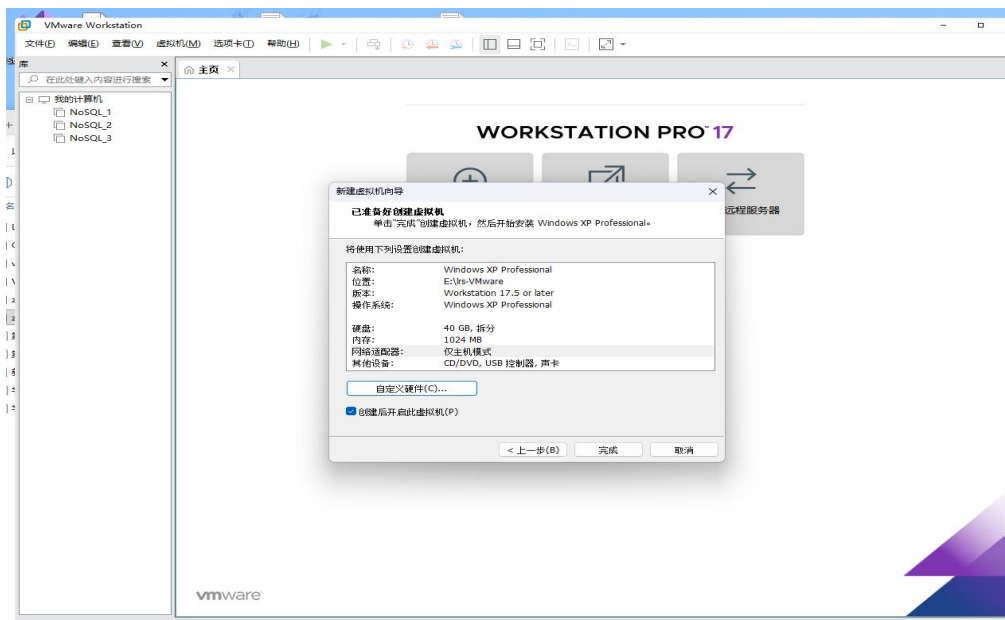
(3) 通过 OllyDbg 将 C 源码与汇编控制流对应起来，理解返回值（EAX）与条件跳转（JZ/JNZ）导致的分支差异。

## 二、实验环境与工具

操作系统：Windows XP Professional Service Pack 3（32-bit）



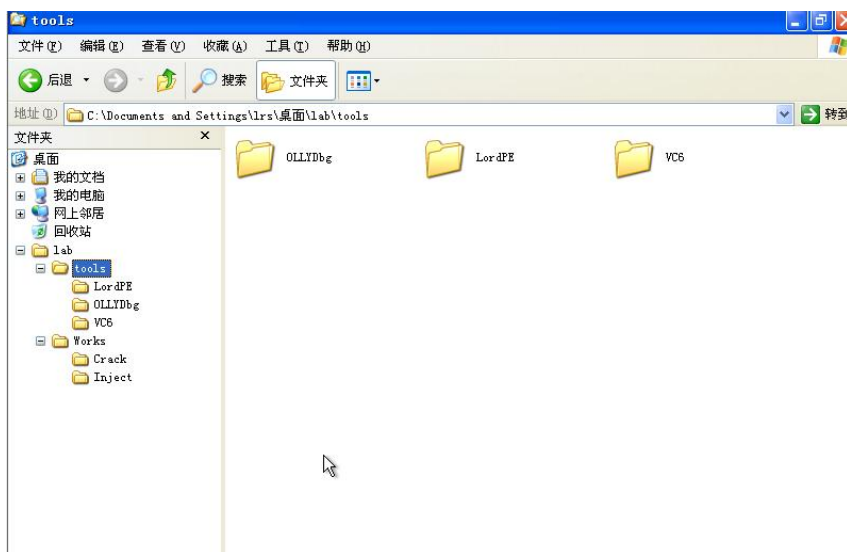
虚拟机平台：VMware Workstation



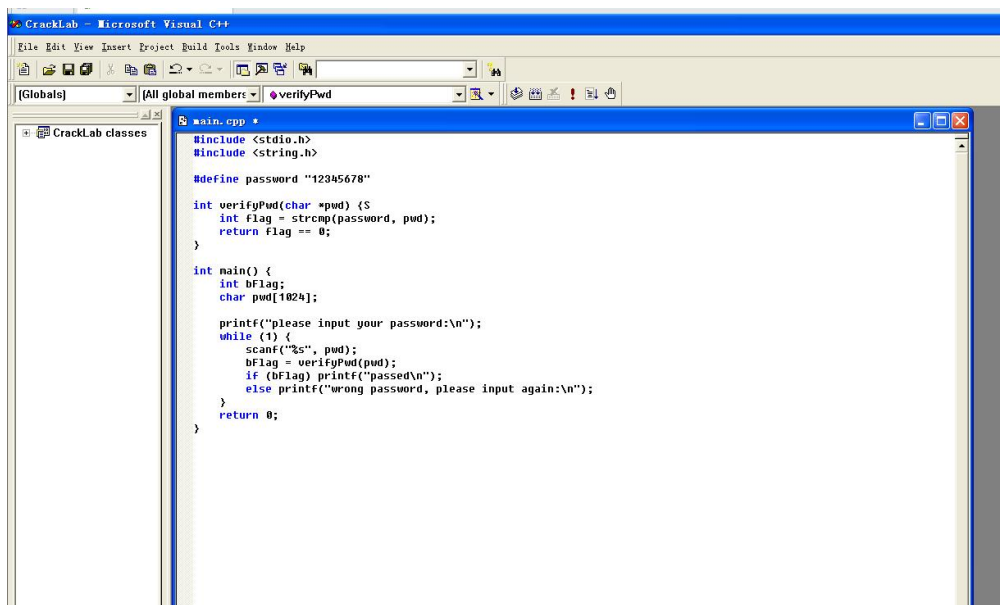
工具：LordPE Deluxe；OllyDbg；Microsoft Visual C++ 6.0（Win32 Debug）

工作目录：

- 实验一：C:\\Lab\\Work\\Inject\\winmine\_patched.exe
- 实验二：C:\\Lab\\Work\\Crack\\CrackLab\\Debug\\CrackLab.exe



编译环境：Microsoft Visual C++ 6.0（VC6，Win32 Debug）

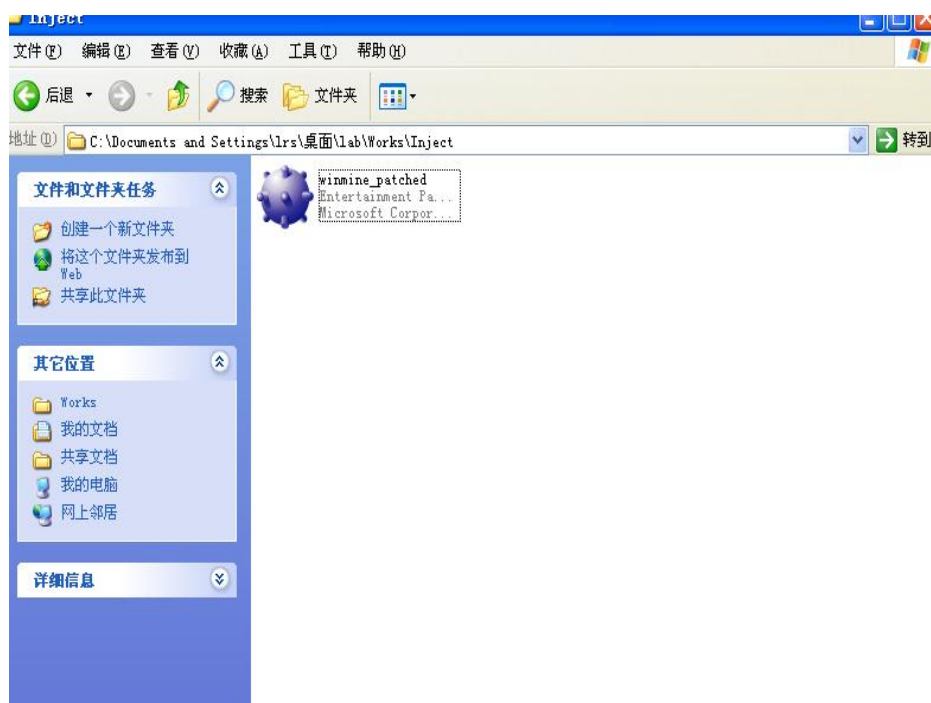


### 三、PE 文件代码注入（winmine\_patched.exe）

#### 3.1 样本准备

将系统自带扫雷程序复制到工作目录后进行实验，避免直接修改系统目录原文件。

样本复制后的路径与文件名

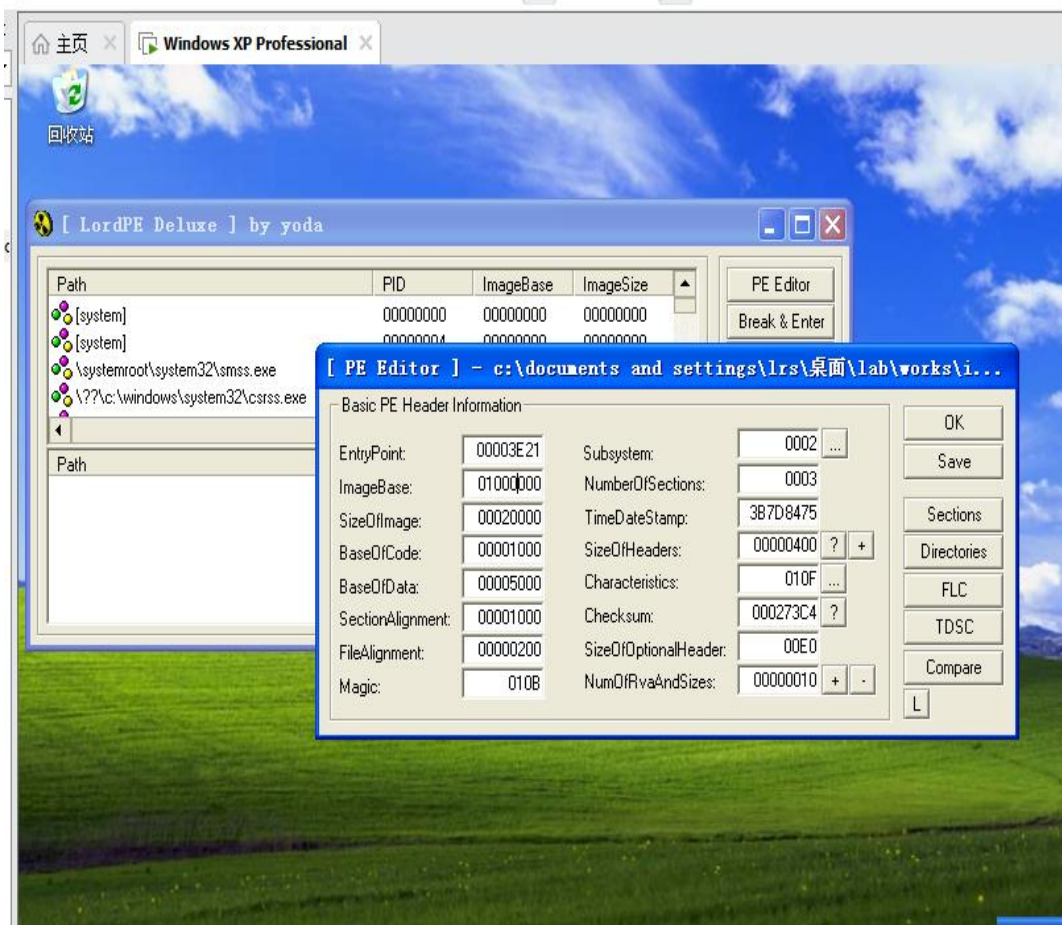


### 3.2 入口点（OEP）定位与验证

(1) 使用 LordPE 的 PE Editor 打开 winmine\_patched.exe，记录 ImageBase 与 AddressOfEntryPoint（RVA）。

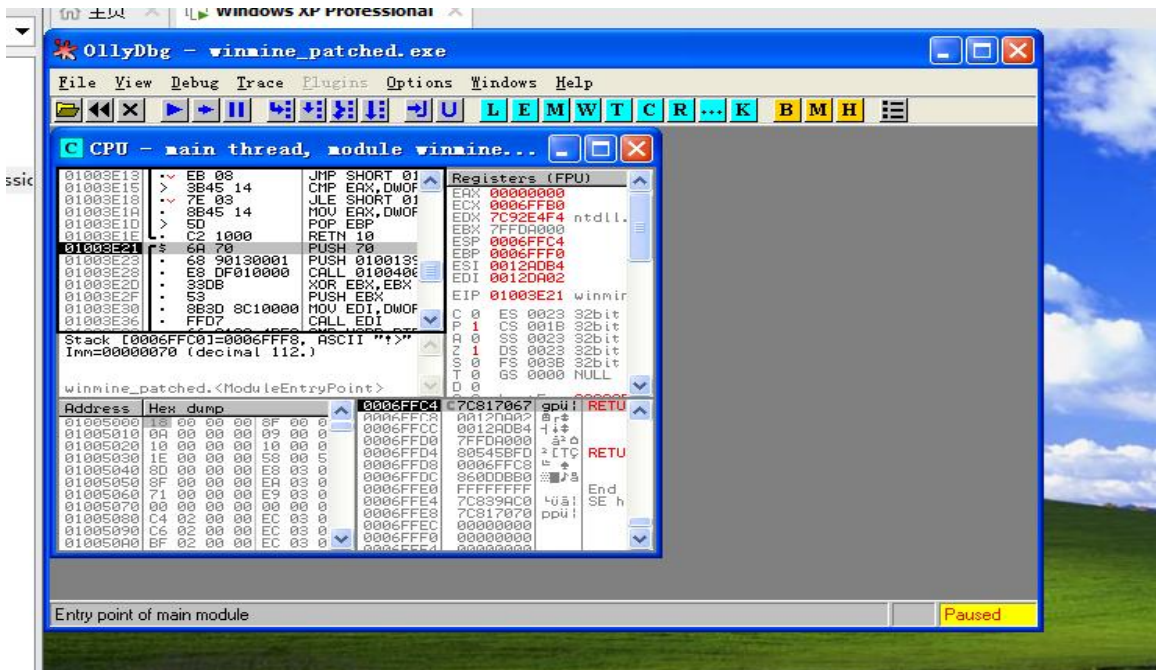
本次记录值：ImageBase = 0x01000000；AddressOfEntryPoint = 0x00003E21；  
因此 OEP(VA)=0x01003E21。

LordPE 显示 ImageBase 与 AddressOfEntryPoint。



(2) 使用 OllyDbg 打开 winmine\_patched.exe，初始暂停在入口点附近，EIP 与 OEP 计算结果一致，用于验证入口点定位正确。

OllyDbg 初始暂停界面（Entry point of main module / EIP=01003E21）。



### 3.3 调用链与参数含义

在 MessageBoxW 调用点附近可观察到典型的“资源字符串加载 → 参数准备 → API 调用”流程：

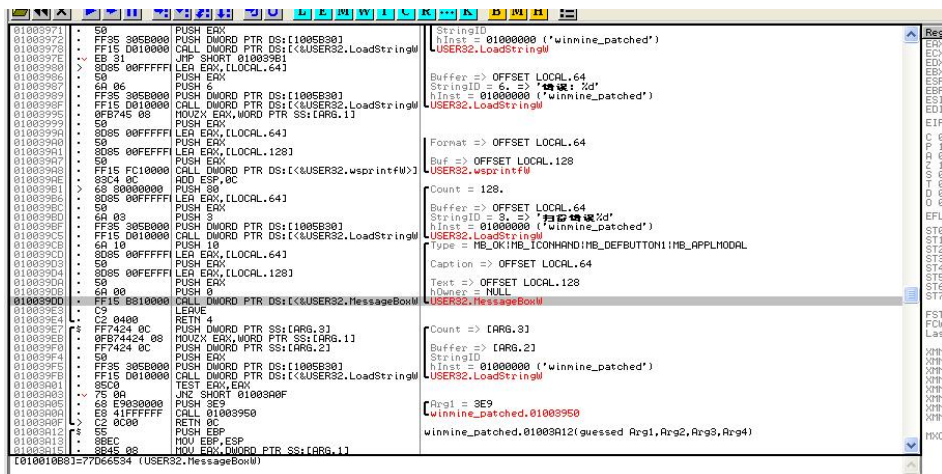
- (1) LoadStringW：从资源表按 StringID 加载宽字符串到缓冲区；
- (2) sprintfW（或同类格式化函数）：拼接/格式化字符串；
- (3) MessageBoxW：弹窗显示。

参数含义（MessageBoxW 原型）：MessageBoxW(HWND hWnd, LPCWSTR lpText, LPCWSTR lpCaption, UINT uType)。

在汇编中通常能看到 push uType / push caption / push text / push hWnd，体现 stdcall 的“右到左压栈”。

call MessageBoxW 前的 push 参数与 call 指令片段：





### 3.4 注入目标与可验证性设计

为保证“可验证”，注入逻辑选择产生可见现象作为成功标志，并要求主程序在现象出现后仍能继续正常执行。

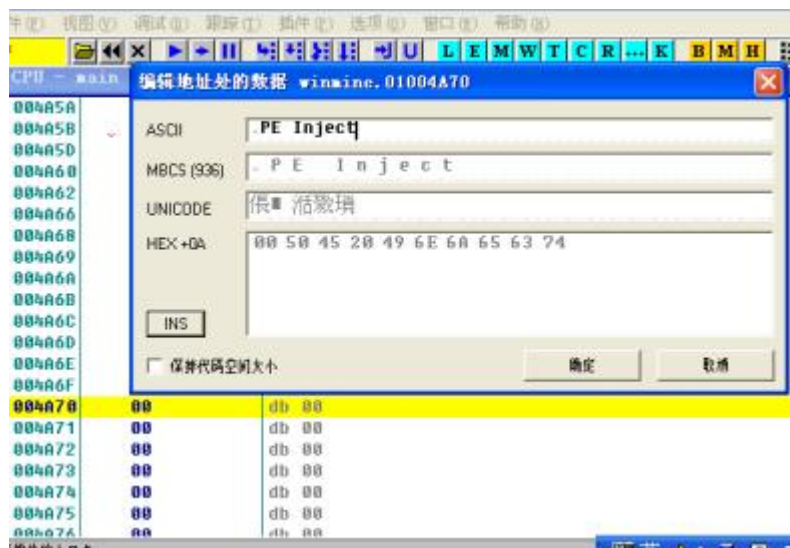
注入逻辑从结构上包含：保存执行现场 → 调用产生可见效果的系统 API → 恢复现场 → 回到原始控制流继续执行。

### 3.5 注入代码片段

(1) 注入代码片段截图应能清晰看到：注入位置地址范围、注入后的新增字节/指令、以及回到原始执行路径的控制流关系。

(2) 注入包含字符串/常量

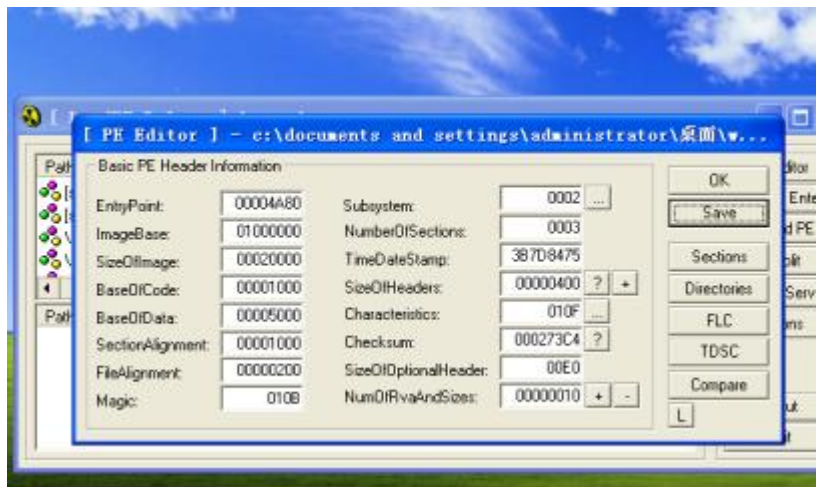
注入的代码片段截图





### 3.6 修改入口点与保存

使用 LordPE 修改 AddressOfEntryPoint，使程序启动时先进入注入逻辑。入口点发生变化。



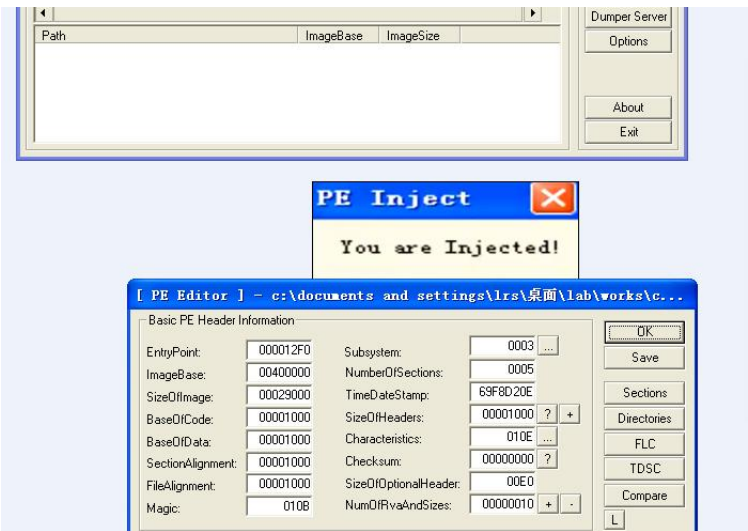
AddressOfEntryPoint 发生变化

### 3.7 运行验证

运行修改后的 winmine\_patched.exe：先出现预期注入现象，==

运行结果如下：

被拒绝



### 3.8 现象与原因分析

入口点决定程序启动后的第一条执行指令。将入口点指向注入逻辑后，控制流首先执行注入段并产生可见现象；随后回到原始控制流继续执行初始化，从而保证

程序正常运行。

注入成功的关键在于：注入段位于可执行区域、调用约定与参数准备正确、且执行后正确恢复现场与返回到原流程。

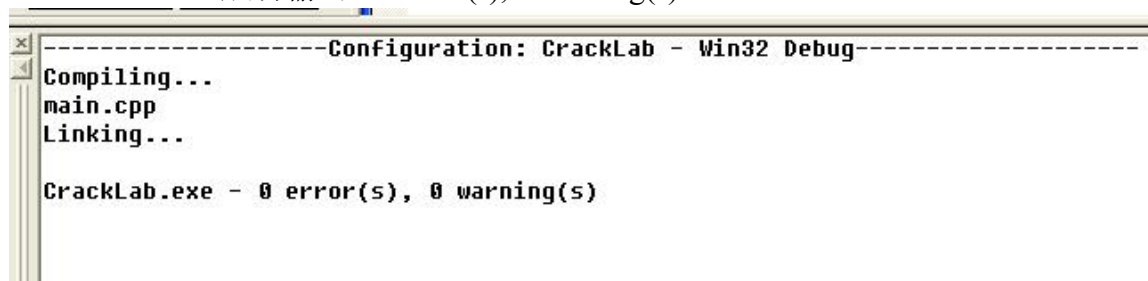
## 实验二

### 四、OllyDbg 调试分析 (CrackLab.exe)

#### 4.1 生成可执行文件 (VC6 Debug)

使用 VC6 新建 Win32 Console 工程 (Debug)，添加 main.cpp 后编译生成 CrackLab.exe。

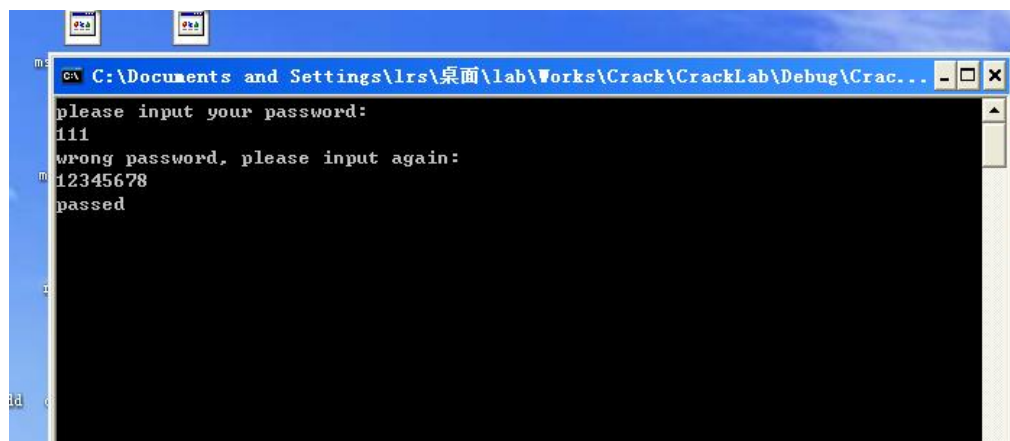
VC6 Build 成功输出 (0 error(s), 0 warning(s))。



#### 4.2 运行现象验证

运行 CrackLab.exe：输入错误密码输出 wrong password；输入正确密码 12345678 输出 passed。

控制台运行结果

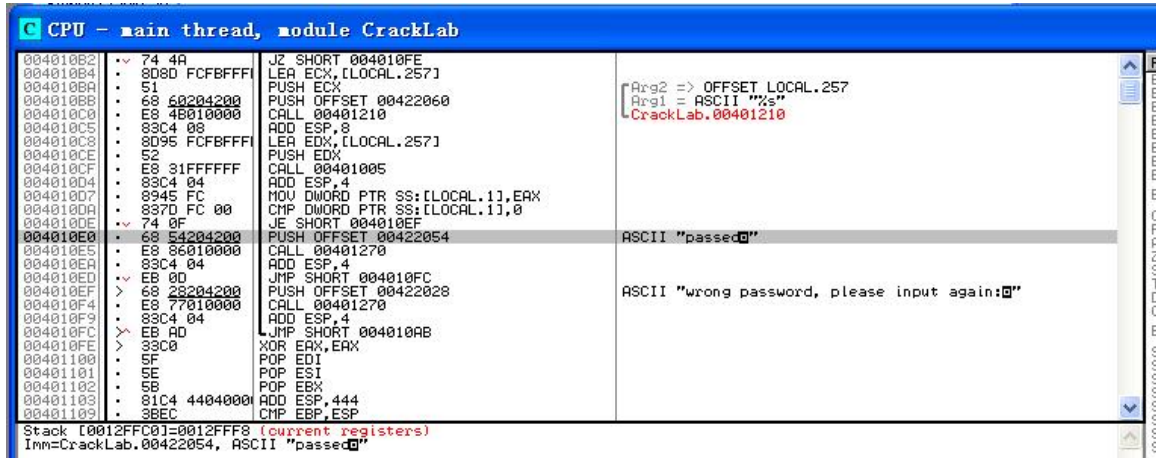


### 4.3 从字符串定位到关键逻辑

通过字符串（passed / wrong password）定位引用位置，跳转到核心逻辑区域，观察到压栈参数、调用比较逻辑函数与分支输出的对应关系。

关键逻辑区域截图（包含压栈参数、CALL、以及与字符串相关联的分支）

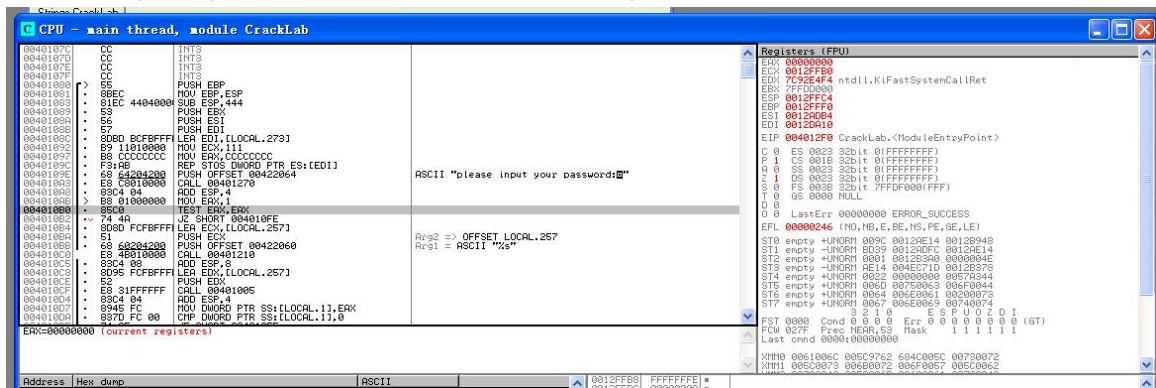
查找 Passed 字符串



### 4.4 返回值（EAX）与条件跳转导致的分支差异

比较/校验函数返回值位于 EAX。TEST EAX,EAX 或 CMP EAX,0 用于判断是否为 0；随后用 JZ/JNZ 选择通过/失败分支。

返回值判断点截图（CALL 返回后 EAX 值 + TEST/CMP + JZ/JNZ）。

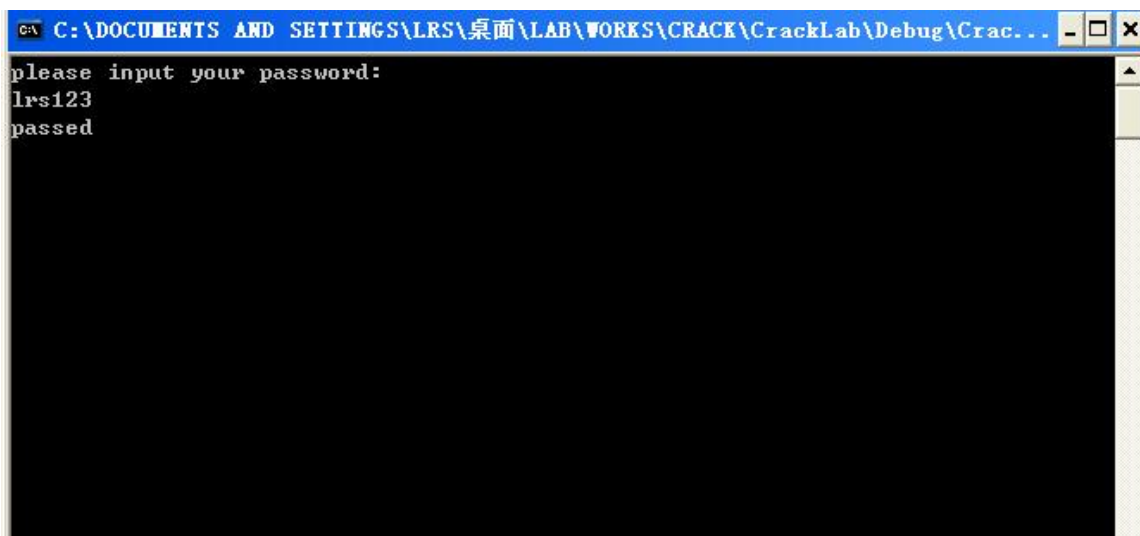


## 4.5 汇编修改前后对比与结果验证

修改对应代码

|    |  |    |             |  |                           |
|----|--|----|-------------|--|---------------------------|
| E4 |  | ·  | 83C4 04     |  | add esp,4                 |
| E7 |  | ·  | 8845 FC     |  | mov byte ptr [ebp-4],al   |
| EA |  | ·  | 8B45 FC     |  | mov eax,dword ptr [ebp-4] |
| ED |  | ·  | 25 FF000000 |  | and eax,000000FF          |
| F2 |  | ·  | 85C0        |  | test eax,eax              |
| F4 |  | ·  | 74 00       |  | jz short 004010F6         |
| F6 |  | ·  | 68 54104300 |  | push offset 00431054      |
| FB |  | ·  | E8 50720000 |  | call printf               |
| 00 |  | ·  | 83C4 04     |  | add esp,4                 |
| 03 |  | ·  | EB 00       |  | jmp short 00401112        |
| 05 |  | >  | 68 28104300 |  | push offset 00431028      |
| 0A |  | ·  | E8 41720000 |  | call printf               |
| 0F |  | ·  | 83C4 04     |  | add esp,4                 |
| 12 |  | >^ | EB A7       |  | jmp short 004010BB        |

输入其他密码也通过



## 五、遇到的问题与解决方法

- (1) VC6 安装入口需从光盘根目录 SETUP.EXE 启动;
- (2) OllyDbg 搜索文本应使用 Strings/All referenced text strings;
- (3) 出现异常现象时优先回到“入口点→执行流→现象验证”的链路进行定位。

## 六、实验总结

- 本次实验围绕“PE 入口点控制流 + IAT 导入调用 + 调试器定位与验证”完成了从静态结构到动态执行的完整闭环。通过 LordPE 读取 ImageBase 与 AddressOfEntryPoint，并在 OllyDbg 中用入口暂停点的 EIP 做交叉验证，进一步加

深了对 RVA/VA、装载基址与 OEP（程序原始入口点）之间关系的理解，也建立了后续分析时“先算地址、再用调试器验证”的规范方法。

- 在导入函数分析方面，掌握了利用 OllyDbg 的跨模块调用/字符串等视图快速定位关键 API 的方法。实验中能从 Intermodular calls 定位到 USER32.MessageBoxW 等调用点，理解了程序通过 IAT 间接调用系统 DLL 的机制，并能解释 MessageBoxA/MessageBoxW 的差异（ANSI/Unicode）以及 Windows 程序中常见的 Unicode 版本调用习惯。这使我在面对没有源码的程序时，能够以“导入函数/字符串”为切入点迅速靠近核心逻辑。

- 在调试分析方面，通过 VC6 生成控制台样例程序并结合 OllyDbg 观察运行流程，建立了“源码语义 ↔ 汇编实现”的对应关系：例如 strcmp 返回值语义（相等为 0）如何映射到寄存器 EAX，以及 TEST/CMP 设置标志位后由 JZ/JNZ 完成 if/else 分支选择。通过对关键调用点、参数压栈与分支跳转的截图与说明，我能够用可复现的方式解释“为什么输入不同密码会得到不同输出”，从而把现象解释落实到指令层面。

- 另外，本次实验也强化了工程化与规范意识：在虚拟机中建立独立工作目录、优先使用副本文件、保存关键数值与截图编号、按评分点组织证据链（入口点 → 关键调用 → 分支判断 → 运行结果），使报告达到“仅看报告即可复现分析过程”的要求。遇到工具使用与环境问题（如安装入口选择、字符串搜索方式等）时，也形成了先定位问题、再用最小验证步骤确认结果的排错习惯。

- 综合来看，本次实验最大的收获是掌握了一套可迁移的逆向分析流程：PE 结构字段定位 → 动态入口验证 → 导入函数/字符串定位逻辑 → 寄存器/标志位解释分支 → 运行现象闭环验证。这套方法不仅适用于本实验中的扫雷与示例程序，也能扩展到其他 Windows 可执行文件的行为理解与安全分析任务中。